SCOOT LEARNING LAB · STUDY GUIDE

What comes after SCOOT? Six experiments, reported honestly

Connected cars, learning agents, max-pressure, broken loops, emissions and speed advice: each idea built, tested in SUMO and measured fairly.

Level 3 · Mastery

INSIDE

- How to test a new traffic idea without fooling yourself
- Why 25% of connected cars was almost enough to replace the loops
- Why a learning agent and max-pressure lost to plain SCOOT here
- How old data rescued SCOOT when 13 detectors broke

Beginner-friendly companion to smrahmanbd.github.io/scoot-learning-lab

The map: SCOOT Mastery

This is the research level, but it is still written for beginners. Each idea is explained with an everyday picture first, then a small hand calculation, then what actually happened when we built it on the lab's SUMO controller. Some ideas did not beat standard SCOOT. That is normal in research, and knowing why is the most useful thing you can learn. There are seven ideas; the first one, how to test fairly, is used by all the others.

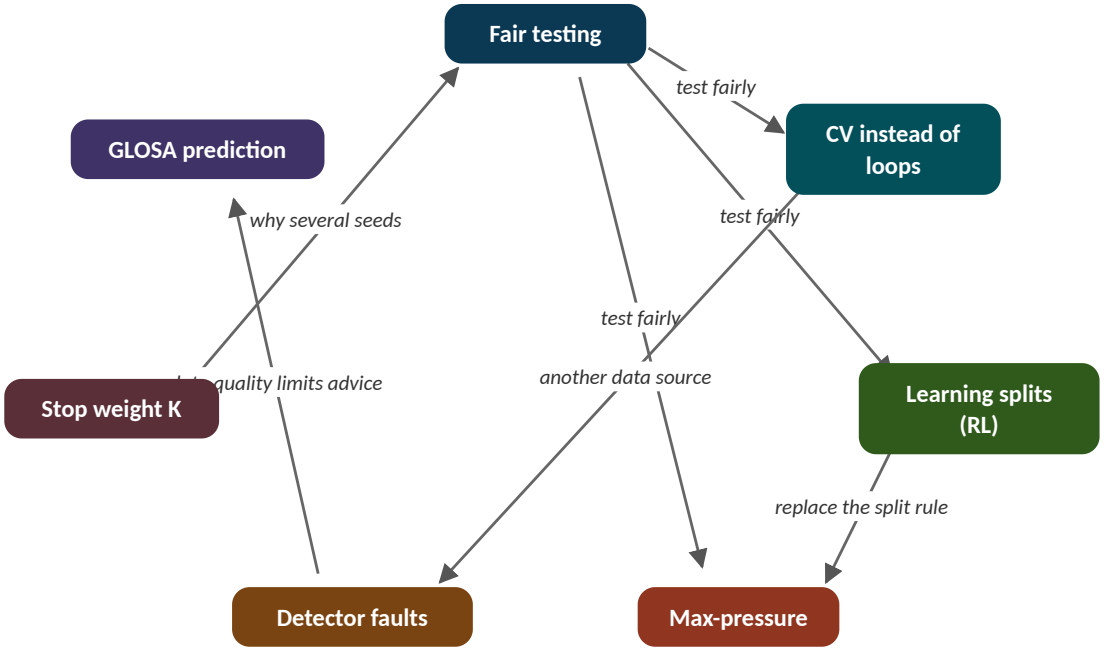


Figure 0. The whole guide on one page. Each box is one concept section; the arrows say how one idea feeds the next. Come back to this page whenever you feel lost.

1. How to evaluate a new SCOOT idea
2. SCOOT on connected-vehicle data
3. A learning split optimiser
4. Max-pressure and spill-back-aware SCOOT
5. Detector faults and historic substitution
6. Emissions-aware SCOOT: the price of a stop
7. Predicting stage changes for GLOSA

HOW TO USE THIS GUIDE

Read the Foundation and Advanced guides first, or at least their cheat sheets: this guide assumes you know LPUs, DoS, the performance index (PI) and the three optimisers. Read one idea per sitting. Do every **Simple calculation** with a pencil. For each study, ask yourself before reading the result: *what do I expect, and why?* Then compare. The **Recap so far** box grows as you go.

What you will be able to do

- Design a fair test of a new SCOOT idea: one change, same demand, several seeds, every second of delay counted.
- Estimate LPUs from connected vehicles and say how the error depends on penetration.
- Describe a learning split optimiser (state, action, reward) and explain why our first one lost.
- Compute a max-pressure-style exit-blocking score and say when it should help.
- Explain what stuck-on and stuck-off detectors do to SCOOT and how historic substitution recovers.
- Show how the stop weight K changes the best decision, and why single runs can mislead.
- Judge whether a predicted end of green is good enough for speed advice (GLOSA).

Before we start

HOOK

Better data was worse. SCOOT running on connected-vehicle counts alone, with every car connected, beat SCOOT on its own loops: 26.3 s against 30.4 s of delay. A loop under a crawling queue over-counts; a simple count does not.

HOOK

The clever agent lost badly. A reinforcement-learning split optimiser averaged 67.7 s of delay on unseen days; plain SCOOT averaged 37.8 s. A negative result, clearly measured, is a real research finding and a starting line.

HOOK

Old data rescued a broken network. When 13 of 40 detectors failed, trusting their readings more than doubled delay. Swapping in each link's own recorded history recovered about 70% of the loss.

Words you will meet

You do not need to memorise these now. Each one is explained again when it first matters.

Word	In plain words
Baseline	The standard method you compare against: here fixed time, actuation and standard SCOOT.
Seed	The number that sets the 'random dice' of a simulation. Different seeds give different but equally likely days.
Ablation	Switching off one part to see how much it was worth.
Connected vehicle (CV)	A car that reports its position by radio or phone. **Penetration** p is the share of cars that do.
Reinforcement learning (RL)	A program that learns which action to take by trial, error and a score (the reward).
Max-pressure	A controller that serves the movement with the biggest upstream queue minus downstream queue.
Spill-back	A queue so long it backs into, and blocks, the junction behind it.
ASTRID	SCOOT's database of past link data, used here as a substitute for broken detectors.
GLOSA	Green Light Optimal Speed Advisory: telling a driver what speed will reach the light on green.

CONCEPT 1 OF 7 · SCOOT MASTERY

How to evaluate a new SCOOT idea

HOOK

It is easy to make a new controller look good: pick a lucky day, measure only what improves, or quietly push queues off the edge of the map. How do you make sure you are not fooling yourself?



IN PLAIN WORDS

Testing a new fertiliser? Grow two identical plants in the same soil and light, change only the fertiliser, and repeat with several pairs. One lucky plant proves nothing. *Where the picture breaks:* traffic 'plants' interact: one early signal decision can change everything that follows, so traffic runs vary even more than plants.

Every study in this level swaps **one** part of SCOOT through a hook in the controller's settings (the data source, the split chooser, the sensor filter, or the stop weight) and keeps everything else standard. Then any difference in results comes from that one change.

All controllers see **identical demand and seeds**. A **seed** fixes the simulation's random dice, so each seed is one 'day'. Because days differ, we run several seeds and report the spread, not the best run.

We report more than delay: stops, bus delay and CO_2 can move in different directions. And we **count every second of delay**, including time vehicles spend waiting to enter the network, so no method can look good by pushing queues off the map. Finally, ask what would stop real deployment: validation effort, explainability, data, and robustness to broken detectors.

$$\text{difference} = \text{new method} - \text{baseline, on the same seed}$$

compare pairs, seed by seed

SIMPLE CALCULATION · Is the difference bigger than the day-to-day spread?

Standard SCOOT on three seeds: 33.1, 38.2, 42.1 s delay. The RL agent on the same seeds: 72.4, 60.8, 69.9 s.

1. SCOOT mean: (33.1 + 38.2 + 42.1) ÷ 3 = 113.4 ÷ 3 = **37.8 s**.
2. SCOOT spread between days: 42.1 − 33.1 = **9.0 s**.
3. Paired differences, RL − SCOOT: 72.4 − 33.1 = 39.3; 60.8 − 38.2 = 22.6; 69.9 − 42.1 = 27.8 s.
4. All three are positive and each is bigger than the 9 s spread.
5. Mean difference: (39.3 + 22.6 + 27.8) ÷ 3 = **29.9 s** worse.

The RL agent is clearly worse, on every day. A 2 s 'improvement' from a single seed would have been inside the 9 s spread and meant nothing.

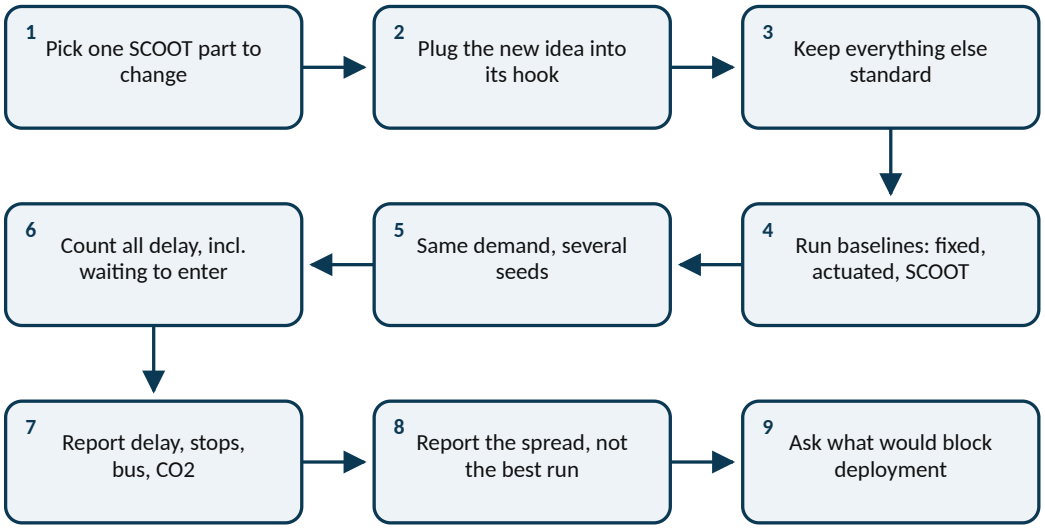


Figure 1. A fair study, step by step. What to notice: the idea enters at one hook only, and the same seeds and demand go to every controller. Read left to right, then down.

TRAP

Do not report your best seed. If the improvement is smaller than the difference between seeds, you have not shown anything yet: run more seeds.

PICTURE THIS · STOP 1 OF THE ROUTE

At the entrance a judge in a white coat holds two identical potted plants, each with three seeds glued on the pot. She changes the label on ONE pot only, then weighs both on a huge brass scale while a vacuum cleaner under the table refuses to suck any queue off the stage.

What it encodes: Two identical plants = same demand; one label = change one thing; three seeds = several seeds; the vacuum that won't hide queues = count all delay.

TRY IT NOW (about 90 seconds)

A new idea gives 35.0 s on seed 7, against SCOOT's 33.1 s, and 36.0 s on seed 8 against 38.2 s. Can you say it is better or worse? *Answer in the key at the back.*

RECAP SO FAR

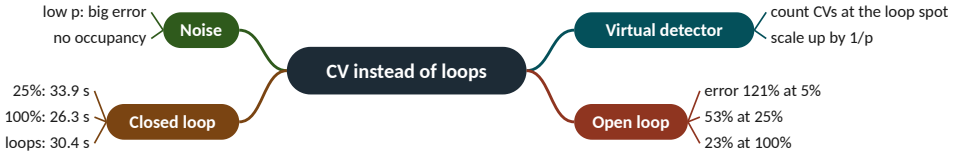
1. **Fair testing: change one thing, same demand and seeds, several seeds with the spread, many measures, count every second of delay.**

CONCEPT 2 OF 7 · SCOOT MASTERY

SCOOT on connected-vehicle data

HOOK

Upstream loops are expensive and break often. Many cars already report where they are. If only one car in four does, can SCOOT still see the traffic?



IN PLAIN WORDS

A pollster asks 1 in 4 voters and multiplies by 4. Ask enough people and the guess is good; ask 1 in 20 and a few odd answers swing the result wildly. *Where the picture breaks:* voters do not move; cars arrive in platoons, so SCOOT's smoothing over several cycles helps more than you might expect.

A **virtual detector** is a line on the map where the loop used to be. Count the connected vehicles crossing it, divide by the penetration p (the share of cars that are connected), and multiply by LPUs per vehicle. That gives an estimated LPU count.

The estimate is noisy when p is small: a few connected cars more or less make a big difference after dividing by p. It also misses occupancy, the second part of an LPU. In the open-loop test (estimating LPUs on identical traffic) one-minute totals were off by **121%** at p = 5%, **53%** at 25% and still **23%** at 100%.

In closed loop (SCOOT actually controlling with CV data) the picture was better, because SCOOT smooths profiles over cycles. Delay was **33.9 s** at 25% against **30.4 s** with loops, and better than loops at 50% and 100% (**27.5** and **26.3 s**): when queues crawl over a loop, occupancy inflates its LPUs, and counts avoid that. Below 10% control degraded clearly (**43.7 s** at 5%).

$$\mathrm{LPU}_{\mathrm{est}} = (\text{connected vehicles crossing}) \div \mathrm{p} \times \text{LPU per vehicle}$$

p = penetration rate (share of connected cars)

$$\text{spread of the estimate} \approx \sqrt{(\mathrm{N}\,\mathrm{p}\,(1-\mathrm{p}))} \div \mathrm{p}$$

N = true vehicles in the period (binomial counting noise)

SIMPLE CALCULATION · How noisy is one cycle's estimate?

20 vehicles cross per cycle. Penetration p = 25%.

1. Expected connected cars: 20 × 0.25 = **5**; estimate 5 ÷ 0.25 = 20. Correct on average.
2. Spread of the CV count: √(20 × 0.25 × 0.75) = √3.75 = **1.94** cars.
3. Spread of the estimate: 1.94 ÷ 0.25 = **7.7** vehicles, about 39% of 20.
4. At p = 5%: √(20 × 0.05 × 0.95) ÷ 0.05 = 0.97 ÷ 0.05 = **19.5**, about 97%.
5. Chance a cycle sees no CV at 5%: 0.95^{20} = **0.36**.

At 5% a third of cycles see no connected car at all. At 25% one cycle is rough, but smoothing over several cycles brings the error down.

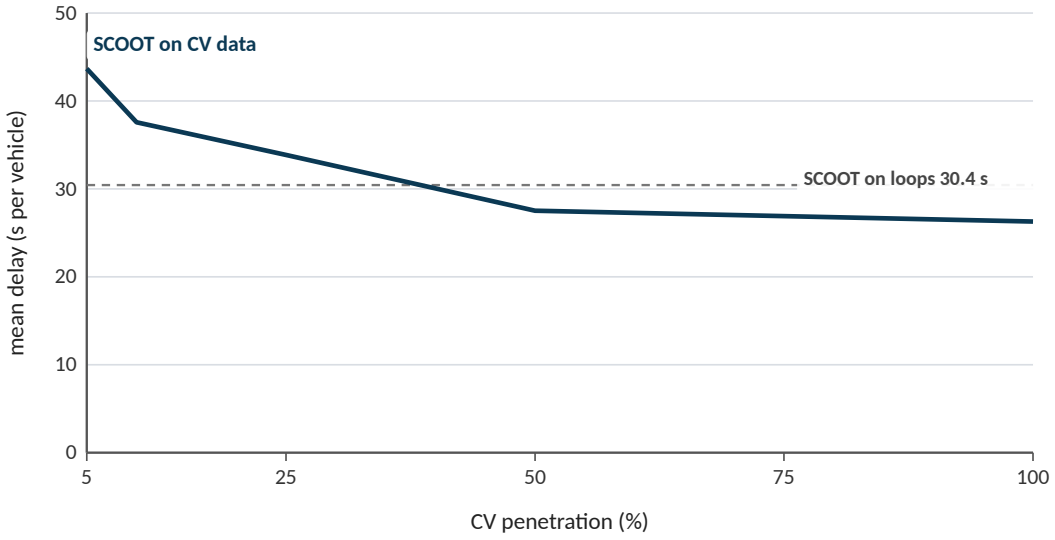


Figure 2. SCOOT's delay on CV data alone, by penetration. What to notice: delay falls as more cars are connected, crosses the loop-based line between 25% and 50%, and is clearly worse below 10%. SUMO region scenario, seed 1.

TRAP

A big open-loop error does not mean bad control. SCOOT smooths and keeps adjusting, so 53% error per minute still gave delay within 11% of loops. Always test in closed loop too.

PICTURE THIS · STOP 2 OF THE ROUTE

At the second stand a crowd walks past a chalk line. Only one in four people carries a glowing phone. A bouncer counts the glowing phones and shouts the number times four through a megaphone. When only one person in twenty glows, his guesses jump wildly and he falls off his stool.
What it encodes: Chalk line = virtual detector; one in four = p = 25% (33.9 s, close to loops 30.4 s); one in twenty = 5%, where control broke down (43.7 s).

TRY IT NOW (about 90 seconds)

With 40 vehicles per cycle and p = 50%, what is the spread of the estimate, in vehicles and as a percentage? *Answer in the key at the back.*

RECAP SO FAR

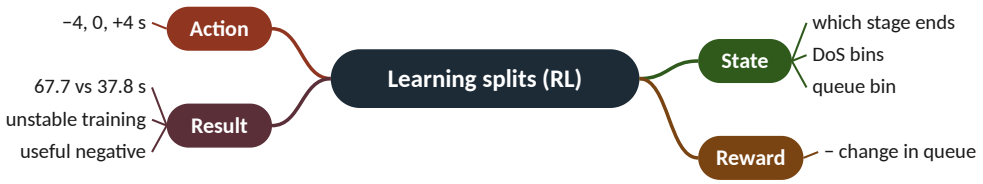
1. **Fair testing:** change one thing, same demand and seeds, several seeds with the spread, many measures, count every second of delay.
2. **CV data: count connected cars at a virtual detector and divide by p; noisy below 10%, near loops at 25% (33.9 vs 30.4 s), better at 50-100%.**

CONCEPT 3 OF 7 · SCOOT MASTERY

A learning split optimiser

HOOK

SCOOT's split optimiser chooses between three moves using a rule written by people. Could a program that learns from trial and error choose better, while keeping all of SCOOT's safety limits?



IN PLAIN WORDS

Teaching a puppy: it tries something, gets a treat or not, and slowly learns. With a clear signal and many tries it learns tricks. With confusing treats and only ten lessons, it just gets excited and knocks things over. *Where the picture breaks:* a puppy sees the whole room; our agent saw only a few coarse 'bins' of the traffic state.

Reinforcement learning (RL) learns a table of how good each action is in each situation. The **state** is a short description of the situation: which stage is ending, the DoS of the ending and next stage (sorted into bins) and the modelled queue. The **action** is SCOOT's own choice: −4, 0 or +4 s on the stage change. The **reward** is minus the change in modelled queue until the next decision.

Only the split choice is learned. Minimum greens, offsets and the cycle optimiser stay as standard SCOOT. That keeps the safety envelope and makes the agent easier to explain and certify than a controller learned from scratch.

What happened. Ten short training runs were unstable (33 to 214 s per episode). On three unseen seeds the greedy agent averaged **67.7 s** against SCOOT's **37.8 s**, with more than twice the bus delay. The state was too coarse, the reward too noisy and the training far too short. The browser toy on the website shows the idea can work on a small problem.

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

Q-learning update: α = learning rate, γ = discount, r = reward

SIMPLE CALCULATION · One Q-learning update

Q(s, +4) = −2.0. The agent takes +4, the queue grows by 1 LPU, so r = −1. In the next state the best Q value is −3.0. α = 0.5, γ = 0.9.

1. Target: r + γ × best next = −1 + 0.9 × (−3.0) = −1 − 2.7 = **−3.7**.
2. Error: target − old = −3.7 − (−2.0) = **−1.7**.
3. Update: −2.0 + 0.5 × (−1.7) = −2.0 − 0.85 = **−2.85**.

The action +4 now looks worse in that state (−2.85), so the agent will try it less. Thousands of such updates are needed; our training had far too few.

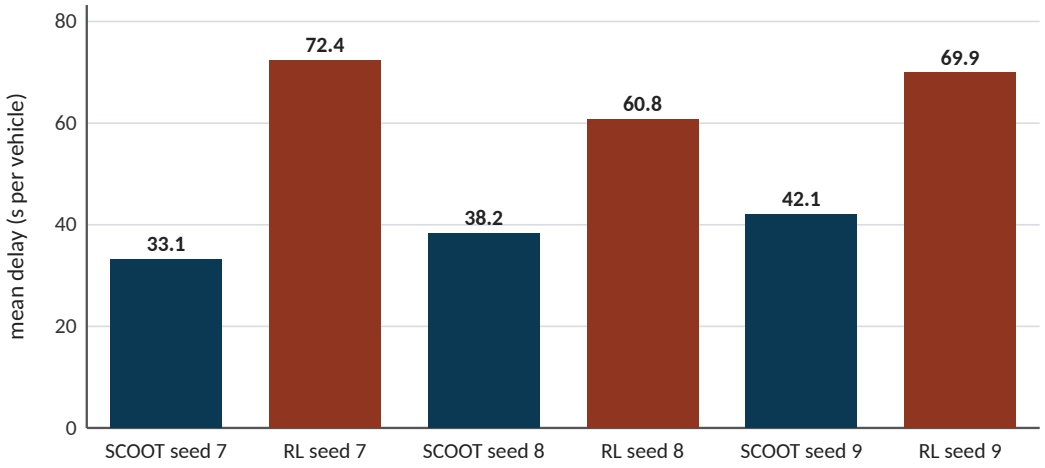


Figure 3. Delay on three unseen seeds: SCOOT and the RL agent. What to notice: the agent is worse on every seed, by more than SCOOT's own day-to-day spread. SUMO region scenario.

TRAP

A learning agent is not automatically better than a rule. Without enough state information, a reliable reward and enough training, it can be much worse, and its failures are harder to explain.

PICTURE THIS · STOP 3 OF THE ROUTE

A clockwork robot puppy has three bones labelled −4, 0 and +4. It grabs a bone, gets a treat or a buzz, and after only ten lessons it is spinning in circles, knocking over a sign reading 67.7, while an old sheepdog calmly herds traffic under a sign reading 37.8.
What it encodes: Three bones = the action set; treat/buzz = reward; ten lessons = too little training; 67.7 vs 37.8 s = the RL agent against SCOOT.

TRY IT NOW (about 90 seconds)

Q(s, 0) = −1.0. The agent holds (0), the queue shrinks by 2 LPU so r = +2, and the best next Q is −1.0. With α = 0.5 and γ = 0.9, what is the new Q(s, 0)? *Answer in the key at the back.*

RECAP SO FAR

1. **Fair testing:** change one thing, same demand and seeds, several seeds with the spread, many measures, count every second of delay.
2. **CV data:** count connected cars at a virtual detector and divide by p; noisy below 10%, near loops at 25% (33.9 vs 30.4 s), better at 50-100%.
3. **RL splits: learn only SCOOT's −4/0/+4 choice (state: DoS and queue bins; reward: −Δqueue); first agent 67.7 s vs SCOOT 37.8 s.**

CONCEPT 4 OF 7 · SCOOT MASTERY

Max-pressure and spill-back-aware SCOOT

HOOK

SCOOT looks at the approach road. Max-pressure also looks at where the traffic is going. If the exit is full, giving green is pointless. Should SCOOT steal that idea?



IN PLAIN WORDS

A doorman letting people into a room looks at the queue outside. A smarter one also checks whether the room is full: if it is, letting more in only jams the doorway. *Where the picture breaks:* max-pressure has no rhythm: it forgets about letting platoons flow through several doors in a row (progression).

Max-pressure (Varaiya, 2013) gives green to the movement with the biggest 'pressure': upstream queue minus downstream queue. It reacts to blocked exits, but it is **acyclic** (no fixed cycle), so it cannot keep green waves between junctions.

The lab also built an **exit-aware SCOOT**: when scoring split options, each link's DoS is divided by (1 + λ × blocking), where blocking measures how far the downstream link's modelled queue is above 30% of its storage. A stage whose exit is filling up looks less urgent.

What happened. Pure max-pressure doubled delay on 300 m links (**68.3** against **30.4 s**) and almost tripled stops. On 120 m links it collapsed into spill-back gridlock (**224.4 s**). Exit-aware SCOOT changed no decisions on 300 m links (no exit filled) and 8 decisions on 120 m links, ending slightly worse (**29.7** against **28.7 s**). SCOOT's own optimisers already kept exits clear here.

$$\text{blocking} = \max(0,\ (q_{\text{down}} \div \text{storage} - 0.3) \div 0.7)$$

0 when the exit is under 30% full, 1 when full

$$\text{score} = \text{DoS} \div (1 + \lambda \times \text{blocking}),\ \lambda = 2$$

used to rank −4 / 0 / +4 split options

SIMPLE CALCULATION · Should stage A get more green?

DoS stage A = 0.95, stage B = 0.80. A's exit link stores 40 vehicles and holds 26. B's exit is empty. λ = 2.

1. A's exit fill: 26 ÷ 40 = **0.65**.
2. Blocking A: (0.65 − 0.3) ÷ 0.7 = 0.35 ÷ 0.7 = **0.5**. Blocking B: **0**.
3. Score A: 0.95 ÷ (1 + 2 × 0.5) = 0.95 ÷ 2 = **0.475**.
4. Score B: 0.80 ÷ (1 + 0) = **0.80**.
5. Plain DoS says A (0.95 > 0.80); exit-aware says **B** (0.80 > 0.475).

Extra green for A would mostly push cars into a filling exit, so the exit-aware rule gives the time to B.

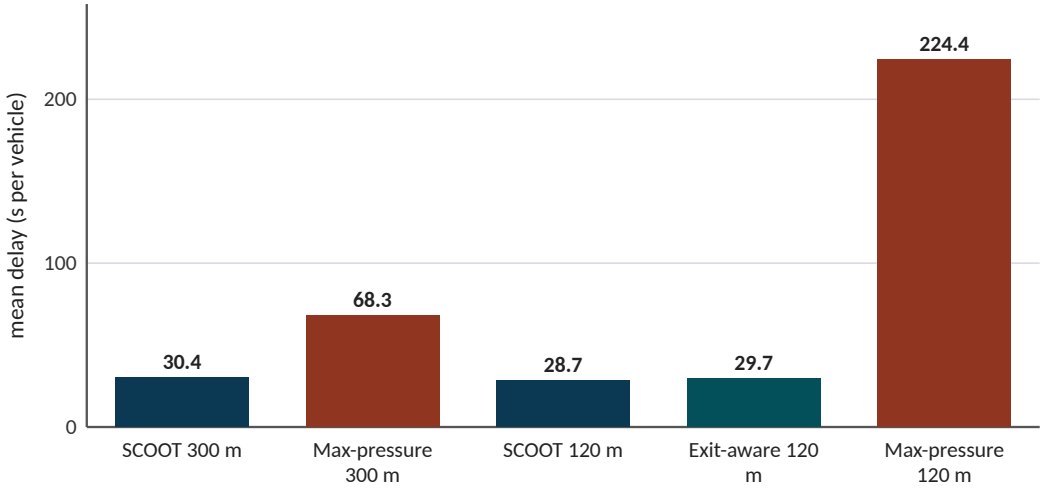


Figure 4. Mean delay: SCOOT, max-pressure and exit-aware SCOOT. What to notice: max-pressure is far worse on both spacings and collapses on short links; exit-aware SCOOT is close to plain SCOOT. Same demand and seed.

TRAP

Max-pressure has strong theory on paper, but it ignores progression and cycle structure. On a coordinated arterial, losing green waves cost far more than its exit awareness gained.

PICTURE THIS · STOP 4 OF THE ROUTE

A wall of pipes with pressure gauges. A plumber opens whichever valve shows the biggest pressure difference, so water squirts in random bursts and never flows smoothly. On the short pipes (120 cm) they burst, soaking a sign reading 224. Next door, a careful plumber divides each gauge by a 'fullness' dial before turning a valve.

What it encodes: Pressure difference = max-pressure; no smooth flow = no progression (68.3 s on 300 m); short pipes bursting = spill-back gridlock (224.4 s at 120 m); fullness dial = exit-aware score.

TRY IT NOW (about 90 seconds)

Same junction, but A's exit holds only 14 of 40 vehicles. Which stage does exit-aware SCOOT favour? *Answer in the key at the back.*

RECAP SO FAR

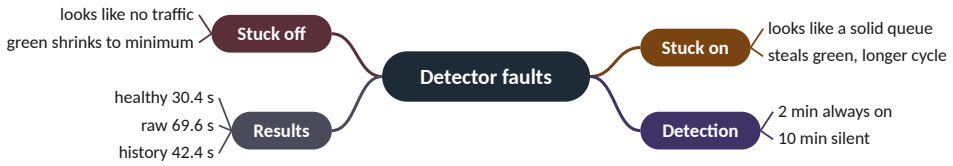
1. **Fair testing:** change one thing, same demand and seeds, several seeds with the spread, many measures, count every second of delay.
2. **CV data:** count connected cars at a virtual detector and divide by p; noisy below 10%, near loops at 25% (33.9 vs 30.4 s), better at 50-100%.
3. **RL splits:** learn only SCOOT's −4/0/+4 choice (state: DoS and queue bins; reward: −Δqueue); first agent 67.7 s vs SCOOT 37.8 s.
4. **Max-pressure: upstream − downstream, no cycle: 68.3 s (300 m), 224 s (120 m) vs SCOOT 30.4 / 28.7 s; exit-aware SCOOT 29.7 s, no gain here.**

CONCEPT 5 OF 7 · SCOOT MASTERY

Detector faults and historic substitution

HOOK

Each SCOOT link depends on one upstream loop. Loops break. A broken loop does not go quiet politely: it can shout 'traffic!' forever. What does that do to the whole network?



IN PLAIN WORDS

If your bathroom scale breaks and says 300 kg every morning, you should not start a crash diet. Better: notice it is impossible and use last month's weight until it is fixed. *Where the picture breaks:* last month's weight is a good guess for you; on an unusual traffic day (an event, a closure) old data can be badly wrong.

A **stuck-on** detector reports a solid queue. SCOOT sees huge demand and congestion, starves the other stages and lengthens the cycle. A **stuck-off** detector reports no traffic, so the link's green shrinks to its minimum while real vehicles queue.

SCOOT's fault logic flags **implausible** detectors. In the lab a detector is flagged after 2 minutes continuously occupied or 10 minutes silent. Then each link's cyclic profile recorded before the failure is used instead, as SCOOT does with its ASTRID database of history.

What happened. 30% of detectors were set to fail at minute 25: 13 of 40, half stuck on, half stuck off. Using their data as if real more than doubled delay (**69.6** against **30.4 s**). Substituting history brought it back to **42.4 s**, recovering about 70% of the loss, with stops slightly below the healthy run.

$$\text{recovered share} = (\text{faulty} - \text{substituted}) \div (\text{faulty} - \text{healthy})$$

1 = all the loss recovered, 0 = none

SIMPLE CALCULATION · How much did history recover?

Healthy 30.44 s, faults with raw data 69.62 s, faults with historic substitution 42.44 s.

1. Loss from faults: 69.62 − 30.44 = **39.18 s**.
2. Gained back by substitution: 69.62 − 42.44 = **27.18 s**.
3. Recovered share: 27.18 ÷ 39.18 = **0.69**, about 70%.
4. Still missing: 42.44 − 30.44 = **12.0 s** per vehicle.

History recovers most, not all, of the damage. Better substitutes (neighbouring links, connected vehicles) could close the remaining 12 s.

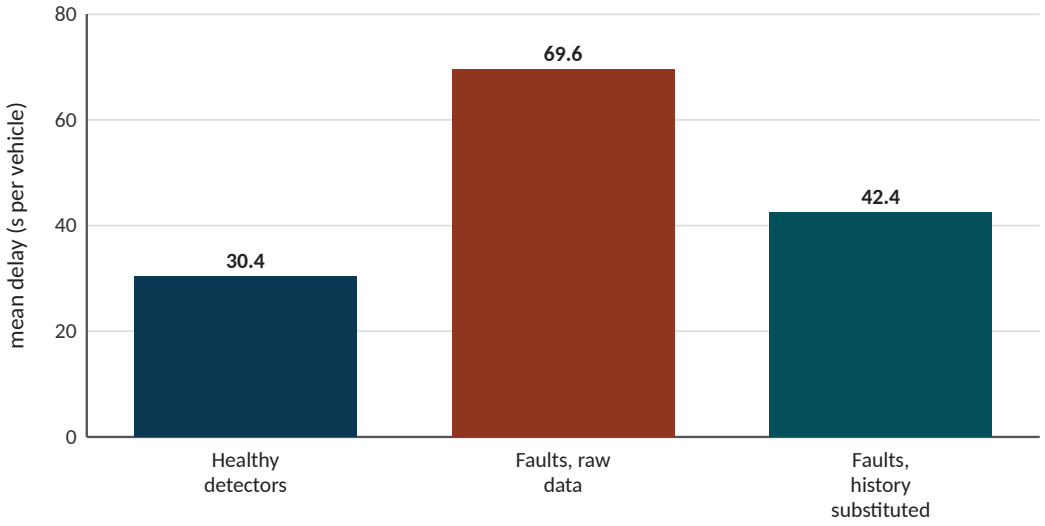


Figure 5. Delay with 13 failed detectors. What to notice: trusting broken detectors costs far more than using last week's pattern. 90-minute region scenario, failures from minute 25.

TRAP

A stuck-on detector is not 'more data'. It is the most harmful fault, because the model believes there is a permanent queue and reshapes the whole region's timings around it.

PICTURE THIS · STOP 5 OF THE ROUTE

A row of 40 bathroom scales; 13 are broken. Some flash 999 forever, others show zero while an elephant stands on them. A librarian with a dusty photo album walks along, covering each broken scale with an old photo of what it used to read. A banner shows 70% in big letters.
What it encodes: 13 of 40 broken = failed detectors; 999 = stuck on, zero = stuck off; the photo album = historic (ASTRID) substitution; 70% = share of the loss recovered.

TRY IT NOW (about 90 seconds)

Suppose a better substitute brought delay with faults to 35.0 s. What share of the loss would it recover? *Answer in the key at the back.*

RECAP SO FAR

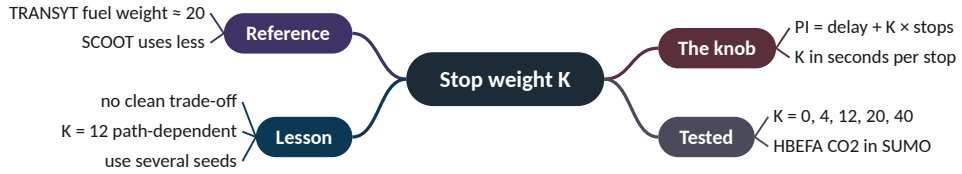
1. **Fair testing:** change one thing, same demand and seeds, several seeds with the spread, many measures, count every second of delay.
2. **CV data:** count connected cars at a virtual detector and divide by p; noisy below 10%, near loops at 25% (33.9 vs 30.4 s), better at 50-100%.
3. **RL splits:** learn only SCOOT's −4/0/+4 choice (state: DoS and queue bins; reward: −Δqueue); first agent 67.7 s vs SCOOT 37.8 s.
4. **Max-pressure:** upstream − downstream, no cycle: 68.3 s (300 m), 224 s (120 m) vs SCOOT 30.4 / 28.7 s; exit-aware SCOOT 29.7 s, no gain here.
5. **Faults: stuck-on and stuck-off loops mislead the model; flag implausible ones and substitute history: 69.6 → 42.4 s (healthy 30.4), ~70% recovered.**

CONCEPT 6 OF 7 · SCOOT MASTERY

Emissions-aware SCOOT: the price of a stop

HOOK

Every stop burns extra fuel to get moving again. SCOOT's performance index already has a knob, K, for how much a stop matters. Turn it up and you should get fewer stops and less CO_2. Did we?



IN PLAIN WORDS

Choosing a route home: you can minimise minutes, or also count each traffic light stop as 'costing' some minutes because stopping is annoying and wasteful. The bigger that cost, the more you prefer a smooth, slightly longer route. *Where the picture breaks:* a single route choice is one decision; SCOOT makes thousands, and an early one can steer the whole run.

The optimisers minimise **PI = delay + K × stops**. K is in seconds of delay per stop. K = 0 ignores stops; a large K prefers smooth flow even if waits get a little longer. TRANSYT's fuel-minimising weighting is about 20 s per stop; SCOOT normally uses much less. SCOOT has offered emissions optimisation since version 4.4.

The lab ran K = 0, 4, 12, 20 and 40 on the region, with SUMO's HBEFA emission model giving CO_2 for every vehicle.

What happened. No clean trade-off. K = 0, 4, 20 and 40 all gave about **30 s** delay, 1.05 to 1.15 stops per vehicle and **227 to 230 g** CO_2 per vehicle. K = 12 was worse on every measure (**42.0 s**, **244 g**) because one early decision sent the cycle optimiser down a different path. Lesson: SCOOT runs are path-dependent; test settings over several seeds.

$$\mathrm{PI} = \mathrm{delay} + \mathrm{K} \times \mathrm{stops}$$

the optimisers pick the option with the lowest PI

$$\text{break-even K} = (\mathrm{delay_B} - \mathrm{delay_A}) \div (\mathrm{stops_A} - \mathrm{stops_B})$$

above this K, the fewer-stops option wins

SIMPLE CALCULATION · Which offset wins as K changes?

Offset A: delay 300 veh·s, 8 stops per cycle. Offset B: delay 320 veh·s, 5 stops.

1. K = 4: $PI_A = 300 + 4 \times 8 =$ **332**; $PI_B = 320 + 4 \times 5 =$ **340**. A wins.
2. K = 20: $PI_A = 300 + 160 =$ **460**; $PI_B = 320 + 100 =$ **420**. B wins.
3. Break-even: $(320 - 300) \div (8 - 5) = 20 \div 3 =$ **6.7 s per stop**.

Below K ≈ 6.7 SCOOT picks the lower-delay offset; above it, the smoother one. K changes decisions, but not always the final results.

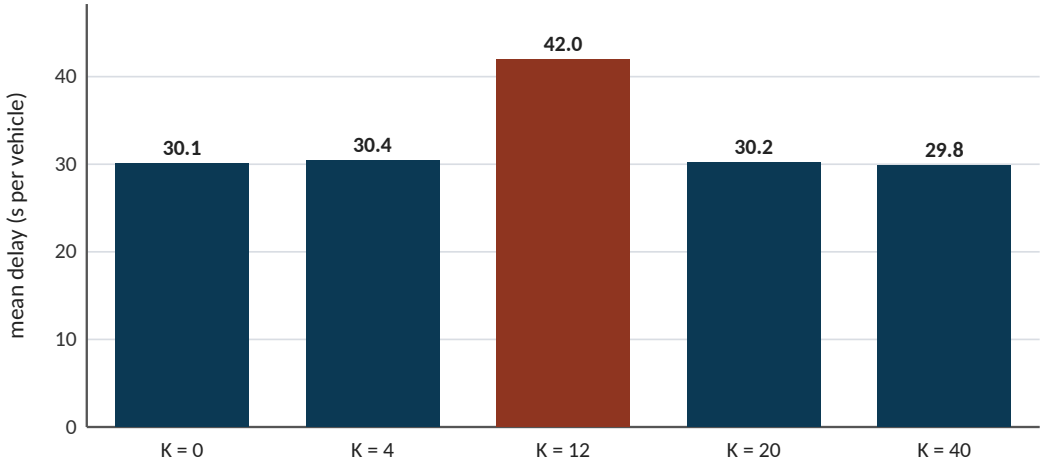


Figure 6. Mean delay for each stop weight K. What to notice: K = 0, 4, 20 and 40 all give about 30 s (and 227-230 g of CO_2 per vehicle); K = 12 is the odd one out on every measure (42.0 s, 244 g), a sign of path dependence rather than a real effect of K. SUMO region scenario, seed 1.

TRAP

Do not conclude 'K = 12 is bad' from one run. When one setting is worse on every measure and its neighbours are fine, suspect path dependence and run more seeds.

PICTURE THIS · STOP 6 OF THE ROUTE

A toy car drives a little track with a dial labelled K. Every time it stops, a balloon inflates on its exhaust. At K = 0, 4, 20 and 40 the balloons are all the same size, about 228 grams. At K = 12 the car takes a wrong turn at the very first corner and a giant balloon pops.

What it encodes: K dial = stop weight; balloon = CO_2 (227-230 g); the wrong first turn at K = 12 = path dependence (42 s, 244 g).

TRY IT NOW (about 90 seconds)

Offset A: delay 250, 10 stops. Offset B: delay 280, 4 stops. Find the break-even K, and the winner at K = 4. *Answer in the key at the back.*

RECAP SO FAR

1. **Fair testing:** change one thing, same demand and seeds, several seeds with the spread, many measures, count every second of delay.
2. **CV data:** count connected cars at a virtual detector and divide by p; noisy below 10%, near loops at 25% (33.9 vs 30.4 s), better at 50-100%.
3. **RL splits:** learn only SCOOT's −4/0/+4 choice (state: DoS and queue bins; reward: −Δqueue); first agent 67.7 s vs SCOOT 37.8 s.
4. **Max-pressure:** upstream − downstream, no cycle: 68.3 s (300 m), 224 s (120 m) vs SCOOT 30.4 / 28.7 s; exit-aware SCOOT 29.7 s, no gain here.
5. **Faults:** stuck-on and stuck-off loops mislead the model; flag implausible ones and substitute history: 69.6 → 42.4 s (healthy 30.4), ~70% recovered.
6. **Stop weight K: PI = delay + K·stops; K = 0/4/20/40 gave ~30 s and 227-230 g; K = 12 was a path-dependent outlier (42 s, 244 g).**

CONCEPT 7 OF 7 · SCOOT MASTERY

Predicting stage changes for GLOSA

HOOK

A car 200 m from the lights could slow gently and arrive just as they turn green, never stopping. But only if someone can say, seconds ahead, when the light will change. Can SCOOT?



IN PLAIN WORDS

A bus timetable that a driver adjusts a little at every stop. Ask 'when does it reach my stop?' a minute ahead and the answer may slip by a couple of minutes; ask when it is one stop away and it is nearly exact, unless something sudden happens. *Where the picture breaks:* here the 'sudden thing' is mostly bus priority, which moves a change at short notice, so the last 10 s are not the most accurate.

GLOSA (Green Light Optimal Speed Advisory) tells a driver which speed will reach the stop line on green. It needs to know when the current green will end. SCOOT keeps a schedule and changes it only in small steps, so its current schedule is a natural prediction.

At every second of every green the lab predicted the end of green from SCOOT's current schedule and compared it with what really happened. The **horizon** is how far ahead the prediction was made. **MAE** (mean absolute error) is the average size of the miss.

What happened. MAE was between **1.3 and 2.6 s** for horizons up to a minute, and 90% of predictions were within **6 s**. The last 10 s had higher error (2.6 s) than 10 s ahead (1.3 s), most likely from bus extensions and recalls; longer horizons add split, offset and cycle changes.

$$\text{advised speed} = \text{distance} \div \text{time until green}$$

for a car arriving before the green starts

$$\text{safety margin} = \text{predicted time left} - \text{time needed}$$

compare with the prediction error

SIMPLE CALCULATION · Can this driver make the green?

Green predicted to end in 15 s. Car 180 m away, speed limit 13.9 m/s (50 km/h). At this horizon: MAE about 1.5 s, 90th percentile about 5 s.

1. Time needed at the limit: 180 ÷ 13.9 = **12.9 s**.
2. Margin: 15 − 12.9 = **2.1 s**.
3. Compare with MAE 1.5 s: on an average day, the driver makes it.
4. Compare with 90th percentile 5 s: 2.1 s is below 5 s, so on a bad day the green ends first.
5. Advice: do not tell this driver to hurry; plan to stop or advise for the next green.

Average accuracy is good, but advice must respect the worst cases. That is why a confidence band is better than a single predicted time.

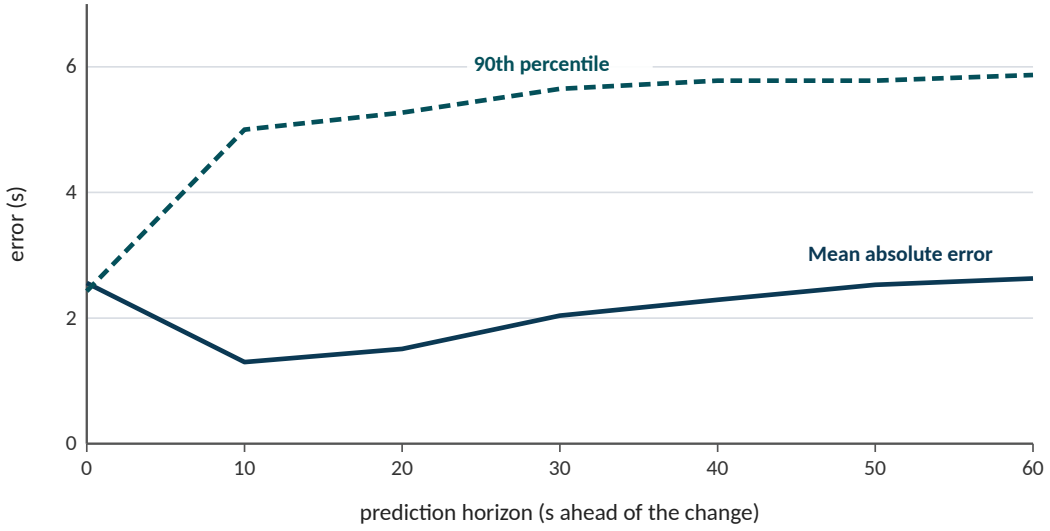


Figure 7. How accurate is the predicted end of green? What to notice: the average error stays small (1.3-2.6 s) out to a minute, while the 90th percentile stays under 6 s. The last 10 s are worse than 10 s ahead.

TRAP

A small average error is not enough for safe advice. Drivers act on single predictions, so the tail (90th percentile, about 5-6 s) decides whether advice is trustworthy.

PICTURE THIS · STOP 7 OF THE ROUTE

In the last tent a fortune teller gazes into a crystal ball shaped like a traffic light and announces 'green ends in 15 seconds!'. She is usually off by only one or two seconds, but a bus keeps barging into the tent at the last moment and she yells 'six!' to cover herself.
What it encodes: Crystal ball = prediction from SCOOT's schedule; 1-2 s = MAE 1.3-2.6 s; the bus = bus extensions at short notice; 'six' = 90th percentile under 6 s.

TRY IT NOW (about 90 seconds)

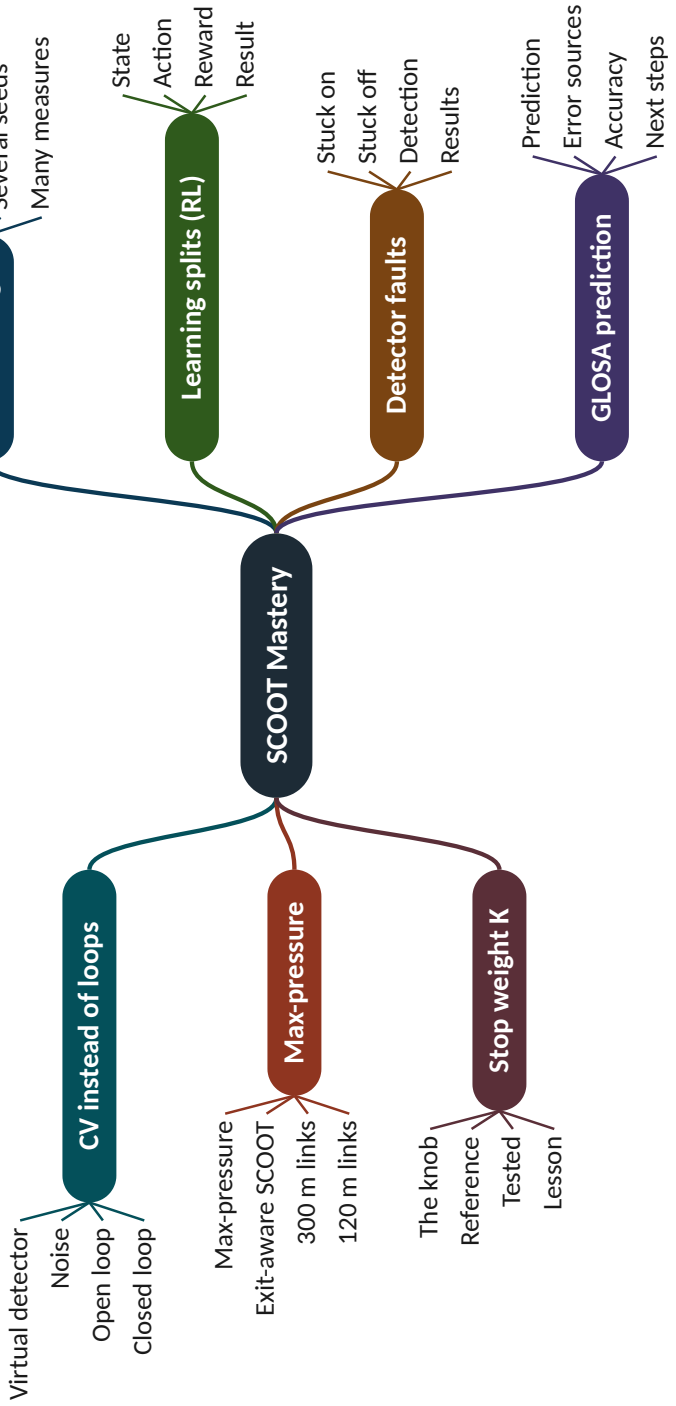
Green predicted to end in 20 s; the car is 220 m away at 13.9 m/s. Margin? Safe to advise 'keep going' if the 90th percentile error is 5.3 s? *Answer in the key at the back.*

RECAP SO FAR

1. **Fair testing:** change one thing, same demand and seeds, several seeds with the spread, many measures, count every second of delay.
2. **CV data:** count connected cars at a virtual detector and divide by p; noisy below 10%, near loops at 25% (33.9 vs 30.4 s), better at 50-100%.
3. **RL splits:** learn only SCOOT's −4/0/+4 choice (state: DoS and queue bins; reward: −Δqueue); first agent 67.7 s vs SCOOT 37.8 s.
4. **Max-pressure:** upstream − downstream, no cycle: 68.3 s (300 m), 224 s (120 m) vs SCOOT 30.4 / 28.7 s; exit-aware SCOOT 29.7 s, no gain here.
5. **Faults:** stuck-on and stuck-off loops mislead the model; flag implausible ones and substitute history: 69.6 → 42.4 s (healthy 30.4), ~70% recovered.
6. **Stop weight K:** PI = delay + K·stops; K = 0/4/20/40 gave ~30 s and 227-230 g; K = 12 was a path-dependent outlier (42 s, 244 g).
7. **GLOSA: predict end of green from SCOOT's schedule; MAE 1.3-2.6 s up to 60 s ahead, 90% within 6 s; advice must respect the tail.**

The master mind map

Every concept and its main branches on one page. Turn the page sideways. Try redrawing it from memory on blank paper, then compare.



Connections and contrasts

Six research studies at a glance

Study	What changed	Baseline result	New result	Verdict
CV data	data source	loops 30.4 s	25%: 33.9 s; 100%: 26.3 s	promising
RL splits	split chooser	SCOOT 37.8 s	67.7 s	not yet
Max-pressure	whole controller	SCOOT 30.4 s	68.3 s (300 m)	worse here
Exit-aware SCOOT	split score	SCOOT 28.7 s (120 m)	29.7 s	needs harder test
Detector faults	fault handling	raw faults 69.6 s	history 42.4 s	recovers ~70%
Stop weight K	PI weight	K = 4: 30.4 s, 227 g	no clean trade-off	use more seeds
GLOSA	prediction	n/a	MAE 1.3-2.6 s	usable with a band

Things beginners mix up

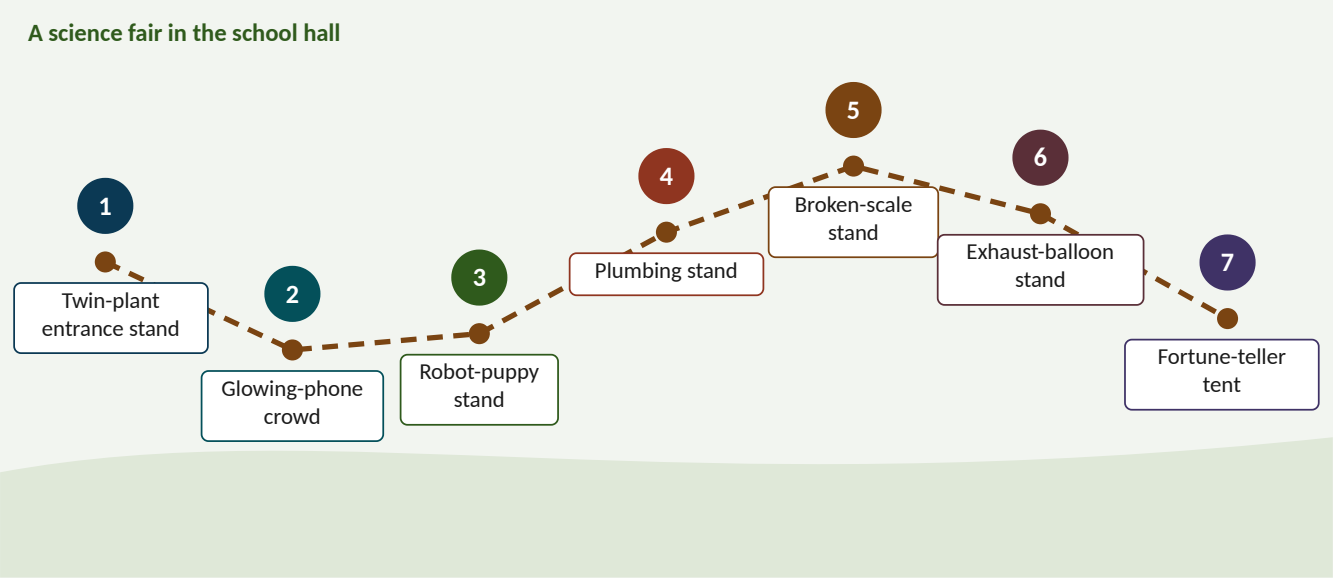
This...	...is not this	The one fact that tells them apart
Open-loop error	**Closed-loop performance**	53% error per minute at 25% CV still gave delay within 11% of loops.
Best seed	**The result**	Report the spread over seeds; SCOOT alone varied by 9 s.
Max-pressure	**Exit-aware SCOOT**	Max-pressure drops the cycle; exit-aware SCOOT only rescales the split score.
Stuck on	**Stuck off**	Stuck on fakes a queue and steals green; stuck off hides traffic and starves it.
Mean error	**Worst-case error**	GLOSA advice must respect the 90th percentile (~5-6 s), not just MAE.

Symbols and terms

Symbol / term	Meaning
p	CV penetration rate: share of vehicles that are connected
LPU_{est}	LPUs estimated from connected-vehicle counts
Q(s, a)	learned value of action a in state s
α, γ	learning rate and discount in Q-learning
λ	weight on exit blocking in the exit-aware score (2)
K	stop weight in PI = delay + K × stops (s per stop)
MAE	mean absolute error of a prediction
p90	90th percentile: 9 in 10 errors are smaller
ASTRID	SCOOT's historic database, used for substitution

The route: A science fair in the school hall

Picture a busy school science fair. You walk past seven stands in order, one per research idea. Each stand has a strange exhibit that holds the idea and its key number. Make the scenes loud and silly.



The memory route. Walk it in order; each stop holds one concept.

Stop 1: Twin-plant entrance stand (How to evaluate a new SCOOT idea). At the entrance a judge in a white coat holds two identical potted plants, each with three seeds glued on the pot. She changes the label on ONE pot only, then weighs both on a huge brass scale while a vacuum cleaner under the table refuses to suck any queue off the stage. *Two identical plants = same demand; one label = change one thing; three seeds = several seeds; the vacuum that won't hide queues = count all delay.*

Stop 2: Glowing-phone crowd (SCOOT on connected-vehicle data). At the second stand a crowd walks past a chalk line. Only one in four people carries a glowing phone. A bouncer counts the glowing phones and shouts the number times four through a megaphone. When only one person in twenty glows, his guesses jump wildly and he falls off his stool. *Chalk line = virtual detector; one in four = p = 25% (33.9 s, close to loops 30.4 s); one in twenty = 5%, where control broke down (43.7 s).*

Stop 3: Robot-puppy stand (A learning split optimiser). A clockwork robot puppy has three bones labelled −4, 0 and +4. It grabs a bone, gets a treat or a buzz, and after only ten lessons it is spinning in circles, knocking over a sign reading 67.7, while an old sheepdog calmly herds traffic under a sign reading 37.8. *Three bones = the action set; treat/buzz = reward; ten lessons = too little training; 67.7 vs 37.8 s = the RL agent against SCOOT.*

Stop 4: Plumbing stand (Max-pressure and spill-back-aware SCOOT). A wall of pipes with pressure gauges. A plumber opens whichever valve shows the biggest pressure difference, so water squirts in random bursts and never flows smoothly. On the short pipes (120 cm) they burst, soaking a sign reading 224. Next door, a careful plumber divides each gauge by a 'fullness' dial before turning a valve. *Pressure difference = max-pressure; no smooth flow = no progression (68.3 s on 300 m); short pipes bursting = spill-back gridlock (224.4 s at 120 m); fullness dial = exit-aware score.*

Stop 5: Broken-scale stand (Detector faults and historic substitution). A row of 40 bathroom scales; 13 are broken. Some flash 999 forever, others show zero while an elephant stands on them. A librarian with a dusty photo album walks along, covering each broken scale with an old photo of what it used to read. A banner shows 70% in big letters. *13 of 40 broken = failed detectors; 999 = stuck on, zero = stuck off; the photo album = historic (ASTRID) substitution; 70% = share of the loss recovered.*

Stop 6: Exhaust-balloon stand (Emissions-aware SCOOT: the price of a stop). A toy car drives a little track with a dial labelled K. Every time it stops, a balloon inflates on its exhaust. At K = 0, 4, 20 and 40 the balloons are all the same size, about 228 grams. At K = 12 the car takes a wrong turn at the very first corner and a giant balloon pops. *K dial = stop weight; balloon = CO_2 (227-230 g); the wrong first turn at K = 12 = path dependence (42 s, 244 g).*

Stop 7: Fortune-teller tent (Predicting stage changes for GLOSA). In the last tent a fortune teller gazes into a crystal ball shaped like a traffic light and announces 'green ends in 15 seconds!'. She is usually off by only one or two seconds, but a bus keeps barging into the tent at the last moment and she yells 'six!' to cover herself. *Crystal ball = prediction from SCOOT's schedule; 1-2 s = MAE 1.3-2.6 s; the bus = bus extensions at short notice; 'six' = 90th percentile under 6 s.*

WALK-THROUGH DRILL

Close the guide. Walk the route in your head, saying each stop out loud and the one number or formula it holds. Then check against the cheat sheet. Repeat until you can do it in under two minutes.

Self-quiz

18 questions: warm-up (recall), core (use the ideas), challenge (combine them). Answer on paper first, then check the key.

Warm-up

Q1. In a fair test of a new SCOOT idea, how many parts of SCOOT should you change at once?

A. One
B. Two
C. All of them
D. As many as needed

Q2. What does a simulation seed control?

A. The network layout
B. The random 'dice' that make each day different
C. The cycle length
D. The detector positions

Q3. How does a virtual detector estimate LPUs from connected vehicles?

A. Counts CVs and multiplies by p
B. Counts CVs and divides by p
C. Measures occupancy of each CV
D. Uses the stop-line loop

Q4. What does a stuck-on detector make SCOOT believe?

A. No traffic
B. A permanent queue
C. A bus is coming
D. The link is closed

Q5. What does GLOSA need to know from the signal controller?

A. The number of lanes
B. When the current green will end (or start)
C. The detector type
D. The CO2 per vehicle

Core

Q6. 20 vehicles per cycle, p = 25%. What is the spread of the CV-based estimate?

A. 1.9 vehicles
B. 5 vehicles
C. 7.7 vehicles
D. 19.5 vehicles

Q7. At 100% penetration, why did CV-based SCOOT beat loop-based SCOOT (26.3 vs 30.4 s)?

A. CVs are faster
B. Loops' occupancy is inflated by crawling queues; counts are not
C. More seeds were used
D. The cycle was fixed

Q8. In the lab's learning split optimiser, what is the action?

A. The whole cycle length
B. −4, 0 or +4 s on the stage change
C. Which detector to read
D. The offset

Q9. Q = −2.0, reward −1, best next Q = −3.0, α = 0.5, γ = 0.9. New Q?

A. −2.85
B. −3.70
C. −1.15
D. −2.00

Q10. Why did pure max-pressure do badly on 300 m links?

A. It ignores the upstream queue
B. It has no cycle, so it loses progression (green waves)
C. Its detectors failed
D. It used CV data

Q11. DoS 0.90, exit holds 33 of 40 vehicles, λ = 2. Exit-aware score?

A. 0.90
B. 0.45
C. 0.36
D. 0.30

Q12. Healthy 30.4 s, raw faults 69.6 s, history 42.4 s. Recovered share?

A. About 40%
B. About 61%
C. About 70%
D. About 100%

Q13. When does historic substitution work worst?

A. On a normal weekday
B. On an unusual day, like an event or closure
C. When few detectors fail
D. When K is large

Q14. Offset A: delay 300, 8 stops; B: delay 320, 5 stops. Which wins at K = 20?

A. A
B. B
C. They tie
D. Cannot tell

Challenge

Q15. K = 12 gave 42 s while K = 4 and K = 20 gave ~30 s. Best conclusion?

A. K = 12 is a bad setting
B. K has a U-shaped effect
C. Likely path dependence; run more seeds before concluding
D. Emissions models are wrong

Q16. Green ends in 15 s (predicted), car needs 12.9 s, 90th percentile error 5 s. Advice?

A. Speed up, you will make it
B. Margin 2.1 s is below the 5 s tail: do not encourage it to go
C. Stop immediately
D. Ignore prediction error

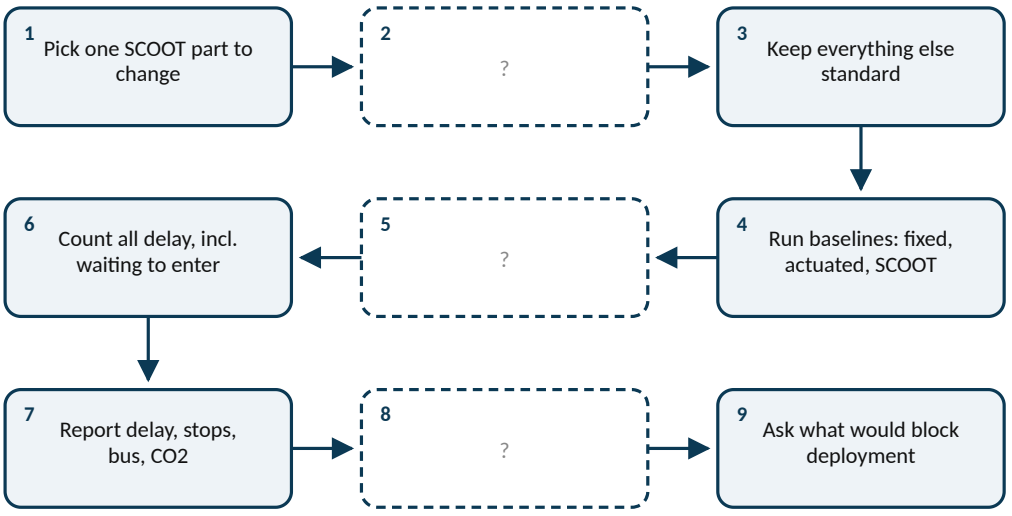
Q17. A new method beats SCOOT by 2 s on one seed. SCOOT varies 33-42 s across seeds. What next?

A. Publish
B. Run more seeds and compare pairs on identical demand
C. Change several more parts
D. Report only the best seed

Q18. Exit-aware SCOOT changed no decisions on 300 m links. Why, and what test would be better?

A. Bug; rerun
B. No exit ever filled; test a grid of short links with spill-back
C. λ was too large; set λ = 0
D. CV data was missing

Draw it. Fill the blank boxes of this flow from memory (it is Figure 1).



Where to go if you missed a question

Missed	Do this
Q1, Q2, Q17	Re-read concept 1: How to evaluate a new SCOOT idea
Q3, Q6, Q7	Re-read concept 2: SCOOT on connected-vehicle data
Q8, Q9	Re-read concept 3: A learning split optimiser
Q10, Q11, Q18	Re-read concept 4: Max-pressure and spill-back-aware SCOOT
Q4, Q12, Q13	Re-read concept 5: Detector faults and historic substitution
Q14, Q15	Re-read concept 6: Emissions-aware SCOOT: the price of a stop
Q5, Q16	Re-read concept 7: Predicting stage changes for GLOSA

Make it stick

BUILD THE SMALLEST WORKING EXAMPLE (15 min)

Design a fair study on paper for this idea: 'give buses a bigger weight in SCOOT's PI'. Write which hook changes, the baselines, the seeds, the measures and what result would convince you. Then do one PI calculation with K = 4 for cars and K = 12 for a bus stop.

Self-check: A good design changes one hook (the PI weights), runs fixed, actuated and standard SCOOT on the same demand with at least 3 seeds, reports delay, stops, bus delay and CO2 with the spread, and counts delay at network entry. Option with 8 car stops + 1 bus stop and delay 300: PI = 300 + 4 × 8 + 12 × 1 = 344.

TEACH IT TO A 14-YEAR-OLD (10 min)

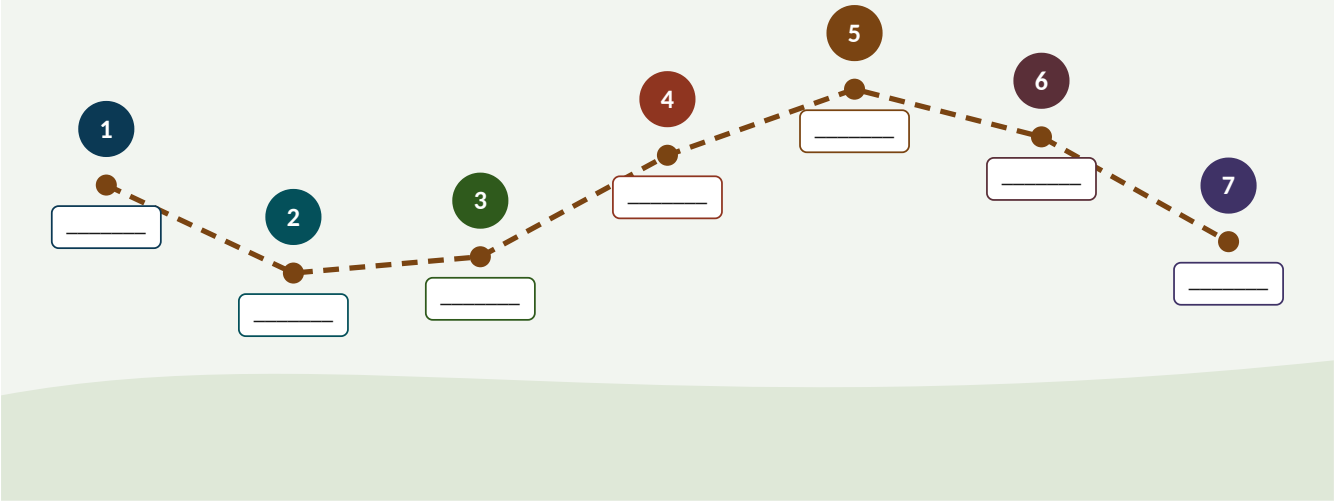
Explain to a 14-year-old why the clever learning computer lost to the older traffic system, and why scientists still count that as a useful result.

Words you may not use: reinforcement, penetration, seed, baseline, percentile

Fill the blank route

Write the concept held at each stop, and its key number.

A science fair in the school hall



One-page cheat sheet

1. How to evaluate a new SCOOT idea

One hook changed; baselines fixed / actuated / SCOOT; identical demand and seeds; report spread; delay + stops + bus + CO_2; hidden (entry) delay counted. SCOOT over 3 seeds: 33.1 / 38.2 / 42.1 s, mean 37.8.

Route: Twin-plant entrance stand

2. SCOOT on connected-vehicle data

LPU_{est} = CV count ÷ p × LPU/veh. Spread ≈ √(Np(1−p))/p. Open loop error 121% / 53% / 23% at 5 / 25 / 100%. Closed loop delay 43.7 / 37.6 / 33.9 / 27.5 / 26.3 s at 5 / 10 / 25 / 50 / 100%; loops 30.4 s.

Route: Glowing-phone crowd

3. A learning split optimiser

State: stage ending, DoS bins, queue bin. Action −4 / 0 / +4. Reward −Δ modelled queue. $Q \leftarrow Q + \alpha(r + \gamma \max Q' - Q)$. Result: RL 67.7 s vs SCOOT 37.8 s (3 unseen seeds), >2× bus delay.

Route: Robot-puppy stand

4. Max-pressure and spill-back-aware SCOOT

Blocking = max(0, (q/storage − 0.3)/0.7). Score = DoS/(1 + λ·blocking), λ = 2. Results: SCOOT 30.4 / MP 68.3 s at 300 m; SCOOT 28.7 / exit-aware 29.7 / MP 224.4 s at 120 m (8 decisions changed).

Route: Plumbing stand

5. Detector faults and historic substitution

Stuck on → fake queue, long cycle; stuck off → minimum green. Flag: 2 min on / 10 min silent. 13 of 40 failed at min 25. Delay: healthy 30.4, raw 69.6, history 42.4 s. Recovered = (69.62 − 42.44)/(69.62 − 30.44) = 0.69.

Route: Broken-scale stand

6. Emissions-aware SCOOT: the price of a stop

PI = delay + K·stops; break-even K = Δdelay/Δstops. TRANSYT fuel weight ≈ 20 s/stop. Results: ~30 s, 1.05-1.15 stops, 227-230 g/veh for K = 0, 4, 20, 40; K = 12: 42.0 s, 244 g.

Route: Exhaust-balloon stand

7. Predicting stage changes for GLOSA

Advised speed = distance ÷ time to green. Margin = predicted − needed; compare with p90. MAE 2.6 / 1.3 / 1.5 / 2.0 / 2.3 / 2.5 / 2.6 s at 0-60 s ahead; p90 ≤ 5.9 s. Bus priority causes late surprises.

Route: Fortune-teller tent

Numbers to remember

- SCOOT over seeds 7/8/9: 33.1 / 38.2 / 42.1 s (mean 37.8)
- CV delay: 43.7 / 37.6 / 33.9 / 27.5 / 26.3 s at 5-100%; loops 30.4
- RL agent 67.7 s vs SCOOT 37.8 s
- Max-pressure 68.3 s (300 m), 224.4 s (120 m)
- Faults: 30.4 / 69.6 / 42.4 s; 13 of 40 failed; ~70% recovered
- K = 0, 4, 20, 40: ~30 s, 227-230 g; K = 12: 42 s, 244 g
- GLOSA MAE 1.3-2.6 s; p90 under 6 s

Answer key

Try it now

Try it 1 (How to evaluate a new SCOOT idea): Differences: +1.9 s and −2.2 s. They point in opposite directions and are far smaller than the 9 s day-to-day spread. **No conclusion yet**; run more seeds.

Try it 2 (SCOOT on connected-vehicle data): √(40 × 0.5 × 0.5) = √10 = 3.16 connected cars; ÷ 0.5 = **6.3 vehicles**, about **16%** of 40.

Try it 3 (A learning split optimiser): Target = 2 + 0.9 × (−1.0) = 1.1. Error = 1.1 − (−1.0) = 2.1. New Q = −1.0 + 0.5 × 2.1 = **0.05**.

Try it 4 (Max-pressure and spill-back-aware SCOOT): Fill 14 ÷ 40 = 0.35; blocking = (0.35 − 0.3) ÷ 0.7 = 0.071. Score A = 0.95 ÷ (1 + 2 × 0.071) = 0.95 ÷ 1.143 = **0.83** > 0.80, so it still favours **A**.

Try it 5 (Detector faults and historic substitution): (69.62 − 35.0) ÷ (69.62 − 30.44) = 34.62 ÷ 39.18 = **0.88**, about 88%.

Try it 6 (Emissions-aware SCOOT: the price of a stop): Break-even = (280 − 250) ÷ (10 − 4) = 30 ÷ 6 = **5 s per stop**. At K = 4: PI_A = 250 + 40 = 290, PI_B = 280 + 16 = 296, so **A** wins (K is below break-even).

Try it 7 (Predicting stage changes for GLOSA): Time needed 220 ÷ 13.9 = 15.8 s; margin 20 − 15.8 = **4.2 s**. That is below 5.3 s, so **not safe** on a bad day; advise caution.

Self-quiz

Q1: A. Change one thing so any difference can be traced to it. (Concept 1)

Q2: B. Same seed = same random arrivals, so controllers face identical days. (Concept 1)

Q3: B. Divide by the penetration p to scale up; multiplying would shrink the count. (Concept 2)

Q4: B. Permanent occupancy looks like a solid queue; stuck-off looks like no traffic. (Concept 5)

Q5: B. Speed advice depends on the timing of the next change. (Concept 7)

Q6: C. √(20 × 0.25 × 0.75) = 1.94 connected cars, ÷ 0.25 = 7.7. 1.9 forgets to divide by p; 19.5 is the 5% case. (Concept 2)

Q7: B. Slow queues over a loop add occupancy LPUs that look like extra demand. (Concept 2)

Q8: B. Only SCOOT's split choice is learned; everything else stays standard. (Concept 3)

Q9: A. Target −3.7, error −1.7, new Q = −2.0 + 0.5 × (−1.7) = −2.85. −3.70 is the target itself. (Concept 3)

Q10: B. Acyclic switching breaks platoon progression; delay doubled to 68.3 s. (Concept 4)

Q11: C. Fill 0.825; blocking (0.825 − 0.3)/0.7 = 0.75; score 0.90 ÷ (1 + 1.5) = 0.36. 0.45 uses blocking 0.5. (Concept 4)

Q12: C. (69.6 − 42.4) ÷ (69.6 − 30.4) = 27.2 ÷ 39.2 = 0.69. (Concept 5)

Q13: B. History assumes traffic resembles the recorded profile; unusual days make it stale. (Concept 5)

Q14: B. PI A = 460, PI B = 420. The break-even is 6.7 s per stop. (Concept 6)

Q15: C. An outlier between two good neighbours, worse on every measure, points to one early decision changing the run's path. (Concept 6)

Q16: B. The average case works but bad days do not; safe advice respects the tail. (Concept 7)

Q17: B. 2 s is inside the 9 s day-to-day spread; paired multi-seed tests are needed. (Concept 1)

Q18: B. Blocking is 0 below 30% fill; the idea needs a network where exits actually fill. (Concept 4)

Draw it: the completed flow is Figure 1.

Your revision plan

When	What	Done
Today	Read one concept a sitting. Do every Try-it-now on paper.	☐
Day 1	Walk the route from memory. Redraw the master mind map.	☐
Day 3	Warm-up and core quiz without notes. Re-read what you missed.	☐
Day 7	Whole quiz. Do the 'build the smallest example' task.	☐
Day 14	Teach-it task out loud. Cheat sheet from memory, then compare.	☐
Before an exam or project	Cheat sheet plus route walk only.	☐

Going further

- Hunt, P.B., Robertson, D.I., Bretherton, R.D. and Winton, R.I. (1981). *SCOOT: a traffic responsive method of coordinating signals*. TRRL Laboratory Report LR1014.
- Department for Transport (1999). *The 'SCOOT' urban traffic control system*. Traffic Advisory Leaflet 7/99.
- Varaiya, P. (2013). Max pressure control of a network of signalized intersections. *Transportation Research Part C*, 36.
- Lopez, P.A. et al. (2018). Microscopic traffic simulation using SUMO. *IEEE ITSC*.
- This lab's website: Mastery page and SUMO studies 1-6, smrahmanbd.github.io/scoot-learning-lab.

HONESTY NOTE

Worked examples use small invented numbers chosen to be easy by hand. All study results come from this repository's SUMO runs (mostly seed 1; RL on seeds 7-9) on a teaching network with illustrative parameters; they show how to evaluate ideas, not how any real SCOOT installation performs. Negative results (RL, max-pressure, exit-aware SCOOT) are first attempts, reported as found.